

Integrating CI into the IRIS Pipeline

Add continuous integration to your IRIS pipeline using GitHub Actions — automatically running evaluation tests, fetching versioned data and models via DVC, and reporting results on every push and pull request.

Mandatory Handout

Review the GitHub Actions commands reference and starter workflow before starting the assignment.

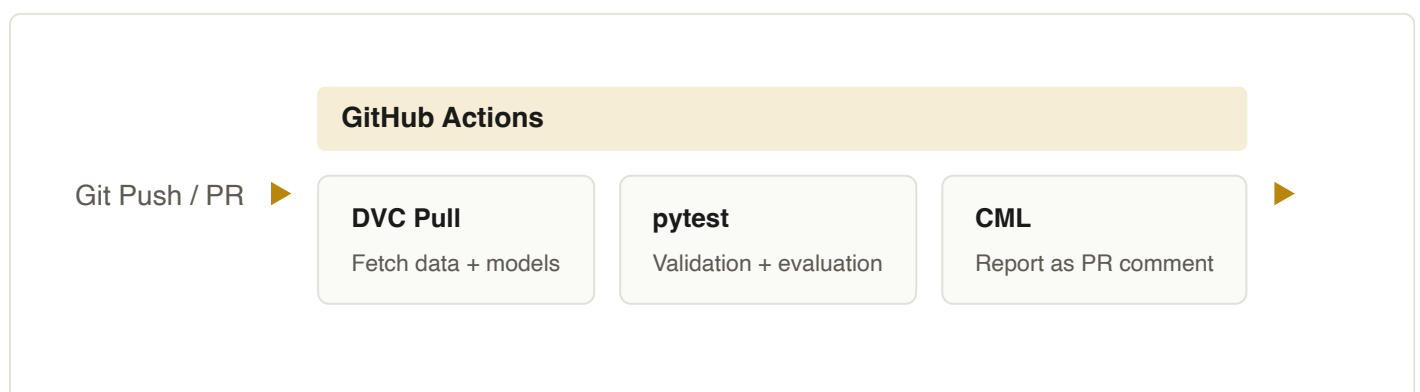
 [GitHub Commands Reference \(branch: week_4\)](#)

Why CI?

In the previous assignments you built an IRIS classification pipeline on Vertex AI, added DVC for data and model versioning, and integrated Feast for consistent feature serving. You now have a reproducible, version-controlled ML workflow — but every time you push code, **nothing verifies whether the pipeline still produces correct results**.

Continuous Integration (CI) closes this gap. By configuring **GitHub Actions** to automatically pull your versioned data and models via DVC, run evaluation and validation tests, and post results back to the repository, you ensure that every change is tested before it reaches your main branch. Combined with **CML** (Continuous Machine Learning), your CI pipeline can even comment test reports directly on pull requests.

Where CI Fits



Key CI Concepts



Familiarize yourself with these building blocks before starting the assignment.

GITHUB ACTIONS

GitHub's built-in CI platform that runs automated workflows in response to repository events like pushes, pull requests, and merges.

WORKFLOW & YAML

A YAML file in `.github/workflows/` that defines the triggers, jobs, and steps for your CI pipeline.

PYTEST

A Python testing framework used to write and run unit tests. In an ML pipeline, it validates data integrity, model outputs, and evaluation metrics.

CML (CONTINUOUS ML)

An open-source tool by Iterative that posts ML metrics, plots, and reports as comments on pull requests — bringing experiment visibility into code review.

CI TRIGGERS

Events that start a workflow — typically `push` and `pull_request`. Each branch can have its own CI configuration triggered independently.

DVC IN CI

Using `dvc pull` inside a GitHub Actions workflow to fetch versioned data and models from your GCS remote, enabling tests to run against the correct artifacts.

GITHUB RUNNERS

The machines that execute your GitHub Actions workflows. GitHub provides hosted runners (e.g., `ubuntu-latest`) with pre-installed tools for building and testing your code.

SECRETS MANAGEMENT

Storing sensitive credentials (GCS service account keys, API tokens) as encrypted `GitHub Secrets` rather than hardcoding them in workflows. Secrets are injected at runtime and never exposed in logs or source code.

BRANCH PROTECTION RULES

GitHub settings that enforce quality gates on protected branches (e.g., `main`). You can require CI status checks to pass and pull request reviews to be approved before a merge is allowed.

Learning Objectives

- Write unit tests for data validation and model evaluation using `pytest`.
- Configure GitHub Actions workflows to run CI on every push and pull request.
- Integrate DVC into CI to fetch versioned data and models for testing.
- Use CML to publish test reports as pull request comments.
- Understand how CI enforces quality gates in an ML pipeline.

Assignment

- Setup CI in git repository `<IITMBS_ROLL_NUMBER>_MLOPS_WEEKLY_ASSIGNMENT` (branch `week_4`)

TASK 1

Write Data Validation Tests

Create unit tests using `pytest` or `unittest` that validate your training and evaluation data — check for expected schema, missing values, correct feature types, and reasonable value ranges.

TASK 2

Write Model Evaluation Tests

Create unit tests that load your trained model, run inference on the evaluation set, and assert that key metrics (accuracy, precision, etc.) meet minimum thresholds. These tests should fail if model quality degrades.

TASK 3

Configure GitHub Actions with DVC

Create a GitHub Actions workflow that checks out the repository, installs dependencies, and uses `dvc pull` to fetch the versioned data and model artifacts from your GCS remote configured in the previous week. The workflow should then run your pytest suite.

TASK 4

Enable CI on Every Push & PR

Configure the workflow triggers so that CI runs on **every push** and **every pull request merge**. Ensure each branch has its own CI execution — the workflow should not be restricted to a single branch.

TASK 5

Report Results with CML

Integrate CML into your GitHub Actions workflow to run a sanity test and publish the test report as a **comment on the pull request**. The report should include test pass/fail status and key evaluation metrics.

TASK 6

Merge to Main via Pull Request

Create a pull request to merge the **week_4** branch into **main/master**. Verify that CI runs automatically on the PR, review the CML report comment, and complete the merge. Demonstrate this entire flow in your video screencast.